

Manual de uso - Llama Desktop Launcher

Versão 0.4.3 · Português (Brasil)

1. O que o aplicativo faz

O Llama Desktop Launcher instala ou localiza o motor `llama.cpp`, encontra modelos locais em formato GGUF e abre a interface do `llama-server` no navegador. Você pode escolher acesso somente local, LAN ou Tailscale. Ao escolher a rede, a API exige uma chave. Os dados trocados com outros computadores trafegam pela rede escolhida.

O instalador do aplicativo **não inclui modelos**. O motor oficial também é baixado na primeira execução, ou você pode indicar uma instalação já existente. A versão do motor testada é `b11514`.

A interface tem três telas: **Llama**, para conversar e iniciar o servidor; **Modelos**, para encontrar GGUF e consultar recomendações; e **Configurações**, para preparar o motor, as pastas e os perfis. A navegação para Configurações permanece disponível em todas as telas.

A versão instalada aparece no cabeçalho, ao lado de “llama.cpp no seu computador”, em todas as telas. Essa é a versão do launcher; a versão do motor llama.cpp é independente.

2. Requisitos

Plataforma / Distribuição	Arquitetura	Pacote oficial / Download direto	Aceleração de GPU
Windows (10 / 11)	x64	`Llama Desktop Launcher Setup 0.4.3.exe`	Vulkan, NVIDIA CUDA, AMD ROCm, Intel SYCL ou CPU
Linux (Debian, Ubuntu, Mint, Pop!_OS)	x64 (amd64)	`llama-desktop-launcher_0.4.3_amd64.deb`	Vulkan, CUDA ou CPU; ROCm/SYCL em x64
Linux (Debian, Ubuntu)	ARM64 (aarch64)	`llama-desktop-launcher_0.4.3_arm64.deb`	Vulkan ou CPU
Linux (Fedora, RHEL, openSUSE)	x64 (x86_64)	`llama-desktop-launcher-0.4.3.x86_64.rpm`	Vulkan, CUDA ou CPU; ROCm/SYCL em x64
Linux (Fedora, RHEL, openSUSE)	ARM64 (aarch64)	`llama-desktop-launcher-0.4.3.aarch64.rpm`	Vulkan ou CPU
Linux (Portátil / Todas as distros)	x64	`Llama Desktop Launcher-0.4.3.ApplImage`	Vulkan, CUDA ou CPU; ROCm/SYCL em x64
Linux (Portátil / Todas as distros)	ARM64	`Llama Desktop Launcher-0.4.3-arm64.ApplImage`	Vulkan ou CPU
macOS (Apple Silicon M1/M2/M3/M4)	ARM64	`Llama Desktop Launcher-0.4.3-arm64.dmg`	Metal
macOS (Intel)	x64	`Llama Desktop Launcher-0.4.3.dmg`	Metal

O espaço necessário depende do modelo escolhido. Reserve espaço para o arquivo GGUF, para o cache e para o motor. Modelos grandes podem exigir muito mais memória do que o tamanho do arquivo em disco. Se o computador não tiver GPU compatível, a execução pode cair para CPU e ficar lenta. No Linux, o pacote oficial do motor é compilado no

Ubuntu; outras distribuições podem precisar de bibliotecas compatíveis. O GitHub Actions compila instaladores oficiais nativos para Windows x64, Linux x64/arm64 e macOS Intel/Apple Silicon.

3. Instalar o aplicativo

O código-fonte está no [repositório do projeto](#). Os links diretos da versão 0.4.3 na tabela acima só funcionarão depois que os instaladores e os arquivos de verificação de integridade ([SHA256SUMS.txt](#)) forem publicados na [Release correspondente do GitHub](#). Para instalar, obtenha o pacote correspondente à sua plataforma na página de Releases ou gere o instalador a partir do código-fonte seguindo o README.

Windows

1. Abra o instalador [Llama Desktop Launcher Setup 0.4.3.exe](#).
2. Escolha a pasta de instalação. O instalador trabalha por usuário, então não precisa colocar o aplicativo em [Program Files](#).
3. Marque ou desmarque a criação do atalho da área de trabalho.
4. Conclua a instalação e abra o aplicativo pelo Menu Iniciar ou pelo atalho.

Linux

- **DEB:** instale pelo gerenciador de pacotes da sua distribuição Debian/Ubuntu.
- **RPM:** instale pelo gerenciador de pacotes da sua distribuição Fedora/RHEL compatível.
- **Applmage:** torne o arquivo executável e abra-o. O Applmage é portátil e fica na pasta que você escolher; não é instalado pelo gerenciador de pacotes.

Os pacotes DEB/RPM usam os diretórios definidos pelo sistema e pelo gerenciador de pacotes. A escolha das pastas de modelos é feita dentro do aplicativo.

Quando o pacote da sua plataforma estiver disponível, use o DEB, RPM ou Applmage correspondente ao seu sistema. Arquivos portáteis experimentais de versões anteriores não substituem esses pacotes.

macOS

1. Abra o arquivo DMG.
2. Arraste o aplicativo para [Applications](#) ou para outra pasta onde você queira mantê-lo.
3. Abra o aplicativo pelo Finder.

As versões de desenvolvimento sem assinatura/notarização podem exigir autorização manual nas configurações de segurança do macOS. Não desative as proteções do sistema de forma geral.

O motor baixado é extraído em uma pasta temporária ao lado do destino final do motor. Isso permite instalar no Linux quando [/tmp](#) e a pasta do usuário estão em sistemas de arquivos diferentes; não é necessário definir [TMPDIR](#). Em caso de falha, o launcher remove essa pasta temporária e mantém o motor configurado anteriormente. O caminho do motor em uso aparece em Configurações.

4. Primeiro uso: motor llama.cpp

No primeiro início, a tela **Configurações** abre automaticamente quando o motor ainda não está pronto, ainda não há modelos, ou não existe um modelo selecionado e salvo. Depois de preparar o motor e escolher um modelo, as próximas aberturas vão para **Llama**. Você pode voltar às **Configurações** pela navegação a qualquer momento. A tela explica o que ainda falta; é possível concluir a configuração depois.

Na seção **Motor llama.cpp**, escolha uma das opções:

1. **Baixar motor oficial:** escolha o **Backend do pacote oficial** antes de clicar no botão. O aplicativo acessa a release [b11514](#) do projeto [ggml-org/llama.cpp](#) no GitHub e baixa o pacote da plataforma e do backend escolhidos. No Windows/Linux, as opções exibidas dependem da arquitetura e incluem Vulkan, NVIDIA CUDA, CPU e, em x64, AMD ROCm e Intel SYCL. No macOS, usa Metal. CUDA baixa também as bibliotecas oficiais correspondentes. O motor é guardado na pasta de dados do seu usuário, fora da instalação do aplicativo.
2. **Escolher pasta existente:** a pasta selecionada precisa conter `llama-server` (ou `llama-server.exe`). Use esta opção para aproveitar uma instalação já preparada, como a pasta [b11193-vulkan](#) do launcher Windows anterior.

O download requer internet. Escolha pelo tipo de GPU, independentemente da marca da CPU: Vulkan funciona com GPUs compatíveis de AMD, Intel ou NVIDIA; CUDA é para NVIDIA, ROCm para AMD e SYCL para Intel. Para uma RTX 4060 no Windows x64, comece por **NVIDIA CUDA 12.4** e mantenha o driver NVIDIA atualizado. ROCm e SYCL podem exigir runtimes e drivers adicionais. Se a release não tiver pacote para a arquitetura detectada, selecione um runtime compilado por você. O aplicativo verifica o SHA-256 quando a API da release fornece o digest. Se você já escolheu um motor e quiser substituí-lo, use **Escolher pasta existente** ou baixe novamente o pacote oficial na mesma tela.

Se você usa o launcher Windows anterior, selecione primeiro a pasta que contém seus GGUF. Em **Configurações > Perfis do launcher anterior**, clique em **Importar perfis TSV** e escolha [launcher-profiles.tsv](#) da pasta do launcher antigo. A importação associa perfis pelo caminho completo do modelo; se você moveu os GGUF, selecione a pasta antiga ou ajuste os perfis manualmente. Perfis já salvos no aplicativo não são sobrescritos. O resultado mostra quantos perfis foram importados, não localizados, já existentes ou inválidos.

5. Pastas e modelos GGUF

O aplicativo aceita **duas localizações** de modelos na tela **Configurações**. A primeira começa sugerida em `~/Models/GGUF`; clique em **Alterar** para escolher outro disco ou pasta. A segunda é opcional e pode ser adicionada ou removida. Você pode mudar as duas pastas depois. A tela **Modelos** procura arquivos `.gguf` dentro das subpastas. Nela, escolha o modelo que deseja usar; o aplicativo guarda a seleção e os ajustes associados ao caminho daquele arquivo.

Mesmo antes de baixar o primeiro modelo, **Modelos** mostra estimativas baseadas na RAM, CPU, GPU/VRAM detectáveis e espaço livre nas pastas de modelos. As faixas são pontos de partida conservadores, não uma garantia de compatibilidade ou desempenho. Confira sempre o tamanho do arquivo GGUF, a licença, os requisitos publicados pelo autor e o espaço disponível. Para MoE, considere o tamanho total dos pesos, não apenas os parâmetros ativos.

Para obter um modelo:

1. Em **Modelos**, explore as opções para abrir o Hugging Face no navegador: **Explorar todos os GGUF** para busca livre sem restrições; **Modelos Quantizados** (onde sugerimos Q4_K_M pelo equilíbrio de memória, mas você pode optar por Q5, Q8, etc.); **Modelos MoE** para arquiteturas esparsas com especialistas; ou **Modelos Densos (sem quantização)** para pesos originais F16/BF16. Você também pode acessar diretamente a busca de [todos os modelos GGUF](#), [modelos quantizados](#), [modelos MoE](#) e [modelos densos F16](#).
2. Escolha um repositório e confira a **licença do modelo**, requisitos de RAM/VRAM, quantização e instruções do autor.
3. Baixe o arquivo `.gguf` para uma das pastas configuradas. Esta versão abre o Hugging Face no navegador; o download do arquivo é feito pelo navegador.
4. Volte ao aplicativo e atualize a lista de modelos. Em **Modelos**, confira a recomendação compatível com seu hardware e selecione o GGUF desejado.

Se um modelo usa projetor multimodal, mantenha um único arquivo `mmproj*.gguf` na mesma pasta. O aplicativo o associa automaticamente. Em modelos divididos em vários arquivos, todos os shards devem permanecer juntos; a lista mostra apenas o primeiro shard.

Atenção: GGUF descreve o formato, mas não garante que todo modelo seja compatível com esta versão do llama.cpp. Repositórios podem exigir uma versão mais nova do motor.

6. Tela Llama: iniciar uma conversa

1. Na lista **Modelo GGUF**, escolha um modelo já detectado nas pastas configuradas. Se ainda não existir um modelo selecionado e salvo, a tela inicial será **Configurações**; baixe ou localize um modelo primeiro na tela **Modelos**.
2. Para conversar dentro do aplicativo, clique em **Conversar aqui**, digite a mensagem e envie. Use Enter para enviar e Shift+Enter para inserir uma linha. O histórico não é salvo.
3. Clique em **Iniciar servidor** para iniciar **llama-server** sem abrir uma página, ou em **Abrir no navegador** para iniciar e abrir a interface web. Com o servidor ativo, o segundo botão apenas abre a interface existente. A tela mostra o endereço de acesso e as sessões configuradas.
4. Ao terminar, clique em **Parar servidor** no aplicativo.

Use **Abrir CLI no terminal** para iniciar **llama-cli** (**llama-cli.exe** no Windows) em um terminal separado, com o modelo e ajustes atuais. Converse e encerre pelo terminal. Pare o servidor ou a conversa integrada antes de iniciar a CLI externa.

O carregamento de um modelo grande pode levar alguns minutos. O registro do motor na parte inferior mostra mensagens e erros. Se a aba do navegador for fechada, o servidor continua ativo até você clicar em **Parar servidor** ou encerrar o aplicativo.

Iniciar o servidor sem abrir o navegador

Clique em **Iniciar servidor** para carregar o modelo e mostrar o endpoint da API sem abrir o navegador. Após o carregamento, **Abrir no navegador** abre a interface do servidor existente sem recarregar o modelo. Use **Parar servidor** para encerrá-lo. Encerre a conversa integrada ou a CLI externa antes de iniciar um servidor.

Para conectar o OpenCode em outro computador ou VM, selecione um endereço LAN ou Tailscale antes de iniciar. Copie o link da API terminado em `/v1` e a chave; o cliente deve enviar a chave como Bearer token. No cliente, consulte `/v1/models`: o identificador será sempre `modelo-local`, independentemente do GGUF selecionado. Configure esse identificador uma vez no OpenCode e defina `model` como `llama-local/modelo-local` para torná-lo padrão. Para trocar de modelo, pare o servidor, selecione outro GGUF e inicie novamente; conversas existentes não são reiniciadas automaticamente. O endpoint e a chave precisam continuar correspondendo ao servidor ativo. Limites de contexto e suporte a ferramentas dependem do modelo escolhido e dos ajustes do servidor. No OpenCode, configure um provedor compatível com OpenAI com esse endpoint e selecione o modelo. Os arquivos do projeto são editados na máquina que executa o OpenCode; a inferência ocorre no servidor. A conversa da CLI sozinha não oferece ferramentas para editar arquivos. No Windows, a CLI externa abre em uma janela visível do PowerShell.

Use **Copiar comando Bash** (Linux/macOS) ou **Copiar comando PowerShell** (Windows) ao lado do endpoint. Cole no terminal, na pasta do projeto. O comando configura o provedor e modelo para aquela execução via `OPENCODE_CONFIG_CONTENT`, sem editar seu arquivo de configuração. Em acesso LAN/Tailscale, ele solicita a chave com entrada oculta; a chave não é incluída no comando copiado. O OpenCode precisa estar instalado. Copie novamente se mudar o endpoint, contexto ou sessões simultâneas. As demais configurações do OpenCode continuam aplicadas, salvo valores de provedor/modelo sobrescritos pela configuração temporária.

Compartilhar acesso na rede interna ou Tailscale

Com o servidor ativo em LAN/Tailscale, use **Copiar link com acesso** para abrir a interface no navegador de outro computador sem digitar uma chave. Use **Copiar Bash com acesso** ou **Copiar PowerShell com acesso** para abrir o OpenCode na pasta do projeto sem pedir a chave. Esses comandos incluem uma credencial temporária de compartilhamento; quem recebê-los poderá usar o modelo.

O link e os comandos compartilhados valem somente para a execução atual. **Revogar acesso compartilhado** invalida todos os anteriores e interrompe conexões compartilhadas existentes; copie novos para conceder acesso novamente. Parar o servidor ou fechar o launcher encerra esse acesso; reiniciar gera uma nova credencial. Isso não revoga a chave permanente da API nem clientes que a usam diretamente. No navegador, o token sai da barra de endereço após a autenticação e fica em cookie HttpOnly.

O compartilhamento usa uma porta adicional de 8181 a 8280; a API original usa 8080 a 8180. No Windows, o botão de firewall libera ambas as portas, limitadas ao executável correspondente e à faixa informada. Linux/macOS exigem liberar as duas portas conforme o firewall local. O OpenCode deve estar instalado no cliente. Prefira Tailscale; uma LAN com HTTP não cifra o conteúdo. Se mudar o modelo, contexto, endereço ou sessões, copie os comandos novamente. Acesso compartilhado autoriza uso do modelo, não acesso automático aos arquivos do servidor.

Conectar outros aplicativos

Em **Conectar aplicativo**, selecione OpenCode, Pi, Hermes, Aider, Continue, Cline, Codex ou Genérico. OpenCode e Aider oferecem comandos Bash/PowerShell: execute na pasta do projeto, com o cliente já instalado. Os demais oferecem **Copiar configuração** e instruções de onde mesclar os dados. O launcher não sobrescreve configurações nem instala clientes.

Para Pi, mescle em `~/pi/agent/models.json` e selecione `llama-local/modelo-local` em `/model`. No Hermes, mescle o bloco `model` em `config.yaml` e execute `hermes chat`. A documentação atual exige ao menos 64000 tokens por sessão para ferramentas; a configuração exporta o contexto real, sem aumentá-lo artificialmente. No Continue, mescle o bloco `models` na configuração YAML existente. No Cline, escolha OpenAI Compatible e preencha Base URL, API Key e Model ID. Genérico fornece esses campos para outros clientes compatíveis.

Copiar configuração com acesso inclui a credencial temporária revogável e o endpoint compartilhado. A configuração direta em rede contém `SUBSTITUA_PELA_CHAVE_DA_API`: preencha com a chave permanente antes de usar. Preserve os providers, modelos e demais ajustes já existentes. Configurações compartilhadas salvas precisam ser atualizadas após revogar ou reiniciar o servidor.

Codex aparece como experimental: a configuração exige Responses API no servidor ou em um adaptador e ainda não foi validada com este motor. Compatibilidade OpenAI não garante suporte a todos os protocolos. Ferramentas, contexto e qualidade de execução dependem do modelo GGUF; suporte a visão não é presumido. O aplicativo trabalha nos arquivos do cliente onde roda.

Integrar APIs de aplicativos: passo a passo

Use **Genérico - OpenAI Compatible** em **Conectar aplicativo** para um aplicativo próprio, serviço, automação ou cliente que aceite OpenAI Chat Completions. O launcher fornece a URL base, chave, ID do modelo e limites; não cria uma API pública na internet.

1. Escolha o endereço antes de iniciar: **Somente este computador** para um cliente no PC servidor; **LAN** para aparelhos na mesma rede; **Tailscale** para aparelhos autorizados na tailnet. Em outro PC, `127.0.0.1` aponta para o próprio cliente, não para o servidor.
2. Clique em **Iniciar servidor** e aguarde aparecer o endpoint. Para LAN/Tailscale, permita a conexão no firewall; o botão do Windows cria as regras para a API direta e o compartilhamento na faixa escolhida.
3. Selecione **Genérico - OpenAI Compatible** e escolha uma das duas modalidades abaixo. Copie a URL mostrada pelo servidor ativo: 8080 e 8181 são exemplos, não portas fixas. A API direta seleciona 8080-8180; a compartilhada seleciona 8181-8280.
4. Configure Base URL com o endereço terminado em `/v1`, API Key com a credencial correspondente e Model ID com `modelo-local`. Se o cliente já acrescenta `/chat/completions`, não acrescente esse caminho à Base URL.

5. Confira primeiro `GET /v1/models`. Depois faça `POST /v1/chat/completions` com JSON e o cabeçalho `Authorization: Bearer CHAVE`. Nos exemplos, a variável de URL já termina em `/v1`; por isso acrescentamos apenas `/models` ou `/chat/completions`.

Acesso temporário ou credencial persistente

Modalidade	Como obter	Validade da credencial
Compartilhada	Copiar configuração com acesso ; usa o endpoint compartilhado e o token da sessão	Até revogar, parar o servidor ou fechar o launcher. Reiniciar exige copiar uma nova configuração.
Direta em rede	Copiar configuração e substituir <code>SUBSTITUA_PELA_CHAVE_DA_API</code> pela chave de Copiar chave ; usa a API direta	A chave é preservada entre reinícios até clicar em Nova chave .
Direta local	Somente este computador , endpoint direto e chave fictícia <code>local</code> para clientes que exigem preencher o campo	A API local não exige autenticação; esse endereço não atende outros PCs.

Para uma integração persistente, mantenha o aplicativo cliente apontando para a **API direta**, com a chave permanente. Não misture a chave permanente com a porta compartilhada nem o token temporário com a porta direta. Guarde a chave no mecanismo de segredos do aplicativo ou no ambiente, fora do código e do repositório. Copiar configuração direta não copia automaticamente essa chave.

Persistência da chave não significa servidor sempre disponível: o launcher e o servidor precisam estar em execução. Depois de reiniciar o PC, abra o launcher e inicie o servidor novamente. Confira se o IP e a porta continuam iguais; atualize a Base URL se mudarem. Para trocar a chave, pare o servidor, clique em **Nova chave**, reinicie e atualize os clientes que usam acesso direto. **Revogar acesso compartilhado** não revoga a chave permanente.

Testar no Linux/macOS com Bash e curl

Cole a Base URL e a chave correspondentes à modalidade escolhida. A chave é solicitada sem exibição. Para acesso direto local, informe `local`.

```
(
  read -rp 'Base URL (terminada em /v1): ' LLAMA_BASE_URL
  read -rsp 'API Key: ' LLAMA_API_KEY
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    "${LLAMA_BASE_URL%}/models"
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    -H 'Content-Type: application/json' \
    "${LLAMA_BASE_URL%}/chat/completions" \
    --data '{"model":"modelo-local","messages":[{"role":"user",
    "content":"Responda apenas: conectado"}],"max_tokens":64,
    "stream":false}'
)
```

O primeiro resultado deve listar `modelo-local`. No segundo, a resposta está em `choices[0].message.content`. Os exemplos usam resposta completa, sem streaming. Clientes com `stream: true` precisam processar Server-Sent Events (SSE).

Testar no Windows com PowerShell

Este exemplo usa o PowerShell e não exige instalar um SDK. Cole a Base URL e informe a chave quando solicitado.

```
& {
  $baseUrl = (Read-Host 'Base URL (terminada em /v1)').TrimEnd('/')
  $secureKey = Read-Host 'API Key' -AsSecureString
  $apiKey = [System.Net.NetworkCredential]::new('', $secureKey).Password
  $headers = @{ Authorization = "Bearer $apiKey" }
  Invoke-RestMethod -Uri "$baseUrl/models" -Headers $headers
  $body = @{
    model = 'modelo-local'
    messages = @(@{ role = 'user'; content = 'Responda: conectado' })
    max_tokens = 64
    stream = $false
  } | ConvertTo-Json -Depth 5
  $response = Invoke-RestMethod -Method Post `
    -Uri "$baseUrl/chat/completions" -Headers $headers `
    -ContentType 'application/json; charset=utf-8' `
    -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
  $response.choices[0].message.content
}
```

Exemplo em Python para seu aplicativo

Requer Python 3. O exemplo usa somente a biblioteca padrão. Antes de executar, defina `LLAMA_BASE_URL` e `LLAMA_API_KEY` no ambiente do processo usando a Base URL e a credencial escolhidas. Não use a chave em parâmetros de URL.

```
import json
import os
from urllib.request import Request, urlopen

base_url = os.environ['LLAMA_BASE_URL'].rstrip('/')
api_key = os.environ['LLAMA_API_KEY']
payload = {
    'model': 'modelo-local',
    'messages': [{'role': 'user', 'content': 'Responda: conectado'}],
    'max_tokens': 64,
    'stream': False,
}

request = Request(
    base_url + '/chat/completions',
    data=json.dumps(payload).encode('utf-8'),
    headers={
        'Authorization': 'Bearer ' + api_key,
        'Content-Type': 'application/json',
    },
    method='POST',
)

with urlopen(request, timeout=120) as response:
    result = json.load(response)
    print(result['choices'][0]['message']['content'])
```

O timeout de 120 segundos é um exemplo; modelos grandes podem precisar de mais tempo. A janela de contexto é a configuração do servidor dividida pelas sessões simultâneas. Prompt, histórico, ferramentas e resposta precisam caber nela; não use o contexto de treinamento como limite da sessão. A resposta da API não executa código nem edita arquivos sozinha: essas ações são responsabilidade do aplicativo cliente.

Diagnosticar a integração por API

Resultado	O que conferir
Conexão recusada ou timeout	Servidor iniciado, modelo carregado, IP/porta atual, mesma rede ou Tailscale ativo e firewall.
401 ou 403	Cabeçalho Bearer presente, chave correta para a modalidade e token compartilhado ainda válido.
404	Base URL termina em <code>/v1</code> ; não há <code>/v1/v1</code> ; o cliente usa Chat Completions. Responses API não foi validada com este motor.
400 ou erro de contexto	JSON válido, ID <code>modelo-local</code> , campos aceitos pelo servidor e limite de contexto por sessão.
Modelo troca mas cliente não reconhece	Inicie o novo GGUF e confira <code>/models</code> ; reinicie a conversa e atualize limites, endpoint ou credencial quando necessário.

Para ferramentas, o GGUF e seu template precisam ser compatíveis com tool calling; alguns motores/modelos exigem um template Jinja adequado. Não habilite visão, ferramentas ou outros protocolos só por o endpoint se declarar OpenAI Compatible.

7. Parâmetros avançados

Acesso, link e sessões simultâneas

Na seção **Acesso e sessões**, escolha **Somente este computador** ou um endereço LAN/Tailscale detectado. Defina de **1 a 8 sessões simultâneas**. O ajuste passa `--parallel` ao `llama-server`; mais sessões compartilham o contexto e a memória disponíveis, portanto comece com 1 ou 2 se o modelo for grande. Esse número é a capacidade simultânea do servidor, não a quantidade de tokens do contexto. O campo **Contexto** (`-c`) controla os tokens.

Ao iniciar em LAN/Tailscale, use **Mostrar chave**, **Copiar chave** ou **Nova chave**. Para trocar a chave, pare o servidor primeiro. O app abre a interface local por uma ponte autenticada. Use **Copiar link da API** para obter o endereço terminado em `/v1`, e **Copiar link da interface** para obter o endereço web na rede. Clientes remotos devem enviar `Authorization: Bearer SUA_CHAVE`; o link por si só não concede acesso. Uma interface remota também precisa oferecer suporte a essa autenticação. Não coloque a chave no URL nem publique a chave em mensagens abertas.

O aplicativo não altera o firewall ao iniciar o servidor. No Windows, com o servidor LAN/Tailscale ativo, informe a **faixa de clientes permitidos** na seção **Firewall do Windows** e clique em **Criar regra para a porta ativa**. Confirme a solicitação de administrador do Windows. As regras limitam cada porta ao executável correspondente (`llama-server.exe` para a API direta e o launcher para compartilhamento) e à faixa indicada. Confira a faixa antes de confirmar; se mudar de porta, crie outra regra. No Linux/macOS, siga as regras do firewall da distribuição. O servidor usa uma porta livre de 8080 a 8180. Prefira Tailscale ou uma LAN confiável, pois o HTTP direto na LAN não cifra o tráfego.

Conversa integrada e CLI externa

Quem prefere o programa original pode clicar em **Abrir CLI no terminal**. O aplicativo abre `llama-cli` (`llama-cli.exe` no Windows) em uma janela de terminal separada, com o modelo e os ajustes atuais. Use o próprio terminal para conversar e encerrá-lo; o botão **Encerrar conversa** controla apenas a conversa interna. Pare o servidor ou a conversa interna antes de abrir a CLI externa. Um terminal gráfico precisa estar instalado no Linux. O terminal externo pode continuar aberto depois que o launcher for fechado.

Os parâmetros avançados aparecem na tela **Llama** dentro do acordeão **Ajustes avançados deste modelo**, fechado inicialmente. Abra-o para revisar os valores, aplicar uma sugestão inicial ou comparar microbatch. Clique em **Salvar**

ajustes deste modelo para guardar o perfil do GGUF selecionado. A tela **Configurações** também oferece o seletor de modelo/perfil e mantém seus ajustes abertos para edição. Os parâmetros são salvos por caminho de modelo.

Aplicar sugestão inicial preenche os campos com base na RAM, no número de threads de CPU e nos dados do GGUF. A estimativa pode incluir GPU/VRAM se o sistema conseguir detectá-las. É um ponto de partida, não uma garantia de velocidade ou de uso exato de memória. Revise os valores e salve o perfil quando quiser mantê-los.

Campo	Argumento do llama.cpp	Como usar
Camadas na GPU	<code>-ngl</code>	<code>auto</code> deixa o motor ajustar; <code>all</code> tenta usar GPU para todas as camadas; <code>0</code> usa CPU.
Dispositivo	<code>-dev</code>	Vazio deixa a escolha automática. Use um identificador como <code>Vulkan0</code> quando houver mais de uma GPU.
Contexto	<code>-c</code>	Número de tokens mantidos na conversa. Valores maiores consomem mais memória.
Camadas MoE na CPU	<code>-ncmoe</code>	Mantém os pesos de especialistas das primeiras camadas na RAM. Só faz sentido para modelos MoE; vazio desativa esse ajuste.
Cache K / V	<code>-ctk / -ctv</code>	<code>f16</code> prioriza fidelidade; <code>q8_0</code> reduz memória com pequeno custo. Outros formatos devem ser testados por modelo.
Flash Attention	<code>-fa</code>	<code>auto</code> , <code>on</code> ou <code>off</code> ; disponibilidade depende do motor/backend.
Threads CPU	<code>-t</code>	Número de threads para partes executadas na CPU. Vazio usa o padrão do motor.
Batch	<code>-b</code>	Tamanho lógico de processamento do prompt.
Microbatch	<code>-ub</code>	Tamanho físico de cada bloco. Não pode ser maior que batch.

O motor recebe também `-fit on` para adaptar a carga à memória disponível. Um modelo denso não deve receber `-ncmoe`; deixe o campo vazio. Para um MoE que não cabe totalmente na VRAM, teste o número de camadas MoE na CPU e observe o desempenho e o consumo de RAM.

8. Analisar recomendações e testar o desempenho

Na tela **Modelos**, o aplicativo prepara um prompt com as informações de hardware disponíveis, o modelo selecionado e seus ajustes. Use **Copiar prompt de análise** para colá-lo em outro lugar, ou **Analisar com modelo selecionado** para enviá-lo pela conversa integrada ao modelo atual. Se ainda não houver modelo, você pode copiar o prompt e usar as sugestões locais; a análise com IA fica disponível depois de selecionar um GGUF.

A resposta do modelo é uma recomendação para você revisar. Ela não altera nem salva os parâmetros automaticamente. Confira compatibilidade, memória, licença e os resultados reais antes de aplicar qualquer mudança. As sugestões locais também são estimativas; o benchmark abaixo mede o desempenho no computador.

Para medir o microbatch depois de selecionar e carregar um GGUF:

1. Na tela **Llama**, abra **Ajustes avançados deste modelo**. Se desejar uma sugestão para vários campos, clique em **Aplicar sugestão inicial**; isso não executa o modelo.
2. Pare o servidor ou conversa ativa e clique em **Comparar microbatch**. O **llama-bench** testa o valor atual e outro de até 128, limitado pelo batch. Quando os dois valores coincidem, há apenas um teste.
3. Aguarde os resultados de processamento de prompt e geração, em tokens por segundo. O aplicativo coloca no campo **Microbatch** o valor com melhor pontuação; os outros campos permanecem como estavam.
4. Revise os campos e clique em **Salvar ajustes deste modelo** para manter o resultado sem iniciar o modelo.

O teste pode carregar o modelo mais de uma vez e levar vários minutos. Ele não salva o perfil sozinho nem otimiza contexto, cache, camadas de GPU ou camadas MoE. Esses valores continuam ajustáveis manualmente.

9. Solução de problemas

Sintoma	O que verificar
Não há modelos na lista	Confirme as duas pastas e a extensão .gguf ; clique em Atualizar lista .
O download do motor falhou	Confira internet, espaço livre e acesso ao GitHub. Tente de novo ou use Escolher pasta existente .
"Pacote sem llama-server"	O arquivo da release pode ter mudado; escolha um runtime compatível manualmente.
O servidor encerra ao abrir	Consulte Registro do motor ; reduza contexto/batch, experimente -ngl auto ou menos camadas de GPU.
GPU não é usada	Confira o backend escolhido, a compatibilidade da GPU e seus drivers; no macOS, confirme o pacote Metal. Verifique mensagens do motor.
Benchmark falha	Confirme que a pasta do runtime inclui llama-bench ; tente parâmetros mais conservadores.
Modelo não carrega	Verifique todos os shards, compatibilidade com b11514 , licença/instruções do repositório e memória disponível.

10. Dados, atualização e desinstalação segura

As preferências e perfis ficam em **settings.json** na pasta de dados do usuário do aplicativo. O motor baixado fica sob **engines/** nessa mesma área. Os modelos permanecem nas pastas que você escolheu. O aplicativo não aceita uma pasta de modelos dentro da instalação nem dentro de sua pasta de dados internos.

Antes de desinstalar, volte ao aplicativo, clique em **Parar servidor** se estiver ativo e feche o aplicativo normalmente. No Windows, o desinstalador verifica se ele ainda está aberto e pede que você o feche; **Tentar novamente** prossegue após o fechamento. Ele não força o encerramento do app. Use **Configurações > Aplicativos instalados**. O desinstalador remove o aplicativo e seus atalhos, mas **preserva por padrão os modelos, perfis e o motor baixado**. Não use opções avançadas de remoção de dados sem verificar o conteúdo da pasta de dados.

No Linux, use o gerenciador de pacotes para remover DEB/RPM, ou apague o arquivo AppImage. No macOS, mova o aplicativo para o Lixo. Em ambos, os dados de usuário e os GGUF permanecem para revisão manual. Se você quiser apagá-los, faça isso separadamente depois de confirmar o caminho e fazer cópia de **settings.json** se desejar preservar os perfis. Nunca apague uma pasta de modelos apenas porque ela foi selecionada no aplicativo.

11. Créditos

O llama.cpp foi iniciado por [Georgi Gerganov](#) e é mantido pelos colaboradores da [ggml-org](#), sob licença MIT. Este launcher independente foi desenvolvido para Célio com Codex, com base no launcher WinForms anterior, e está sob **licença MIT**, com copyright de Célio. Os instaladores exigem publicação separada na Release correspondente para download direto. Os autores do llama.cpp não participam nem endossam este aplicativo. Cada modelo tem licença própria.